

GitHub 협업

초보자 실전 가이드

Repository 만들기부터 Clone, Branch, Commit, Push, Pull Request, Merge 까지

이 문서 하나로 익히는 전체 흐름

GitHub 저장소 만들기 -> Collaborator 초대 -> clone -> main 최신화 -> 내 branch 생성 -> 작업 -> add -> commit -> push -> Pull Request -> Merge -> local main pull

대상 Git/GitHub 협업을 처음 해보는 사람

권장 환경 Git + GitHub + VS Code + 터미널

핵심 원칙

"main 은 함께 쓰는 기준선, 각자 branch 에서 작업하고 PR 로 합친다."

START HERE

1. 먼저 이것부터 이해하기

Git 과 GitHub 의 역할

Git 은 내 컴퓨터에서 코드 변경 이력을 관리하는 도구이고, GitHub 는 그 Git 저장소를 인터넷에서 공유하고 협업하는 공간입니다.

Local 과 Remote

```
[GitHub / Remote Repository]
      ^   |
      |   |
push  | pull
      |   v
[내 컴퓨터 / Local Repository]
```

Local = 내 컴퓨터에 있는 Git 저장소

Remote = GitHub 에 있는 원격 저장소

협업에서는 branch 를 나눠서 작업

```
main
|
+-- sujin-login
+-- minsu-db
+-- jiwon-ui
```

main 은 팀이 함께 사용하는 기준 브랜치입니다. 각자 작업 브랜치를 만들고, 작업이 끝나면 Pull Request(PR)로 main 에 병합합니다.

초보자가 가장 많이 헷갈리는 3 가지

1) add 는 GitHub 업로드가 아님 2) commit 도 아직 Local 기록 3) push 를 해야 GitHub 에 올라갑니다.

CONCEPTS

2. 핵심 용어를 한 번에 정리

용어	정확한 의미	쉽게 기억하기
Repository (Repo)	프로젝트와 Git 이력을 저장하는 공간	프로젝트 저장소
Local	내 컴퓨터의 저장소	내 PC
Remote	GitHub 의 저장소	온라인 공유본
Branch	독립적으로 작업하는 가지	내 작업 공간
origin	clone 한 원격 저장소의 기본 별명	GitHub 주소의 별명
Commit	변경사항을 하나의 기록으로 저장	세이브 포인트
Push	Local 의 commit 을 Remote 로 전송	내 PC -> GitHub
Pull	Remote 최신 commit 을 Local 에 반영	GitHub -> 내 PC
Pull Request	내 branch 를 main 에 합쳐달라는 요청	검토/병합 요청
Merge	서로 다른 branch 의 변경사항을 합침	코드 합치기
Conflict	같은 부분을 다르게 수정해 자동 병합 불가	충돌

Collaborator 방식

하나의 Remote Repository 를 팀원들이 공유하고, 협력자 권한을 받은 사람이 자신의 branch 를 만들어 push 하는 방식입니다. 소규모 협업 실습에 적합합니다.

3. 처음 한 번만 하는 Git 설정

3-1. Git 설치 확인

```
git --version
```

버전이 출력되면 Git 이 설치되어 있습니다.

3-2. 커밋 작성자 이름과 이메일 설정

```
git config --global user.name "sujin"  
git config --global user.email "example@gmail.com"
```

왜 하나요? commit 에는 "누가 작성했는지"가 기록되므로 작성자 정보를 설정합니다.

3-3. 설정 확인

```
git config --global --list
```

주의

이 설정은 컴퓨터 전체에 적용되는 기본값입니다. GitHub 계정 이메일과 맞추면 커밋 연결을 확인하기 편합니다.

3-4. 터미널에서 명령어를 쓰는 기본 규칙

- 명령어를 한 줄 입력하고 Enter 를 누릅니다.
- 오류가 나면 같은 명령을 반복하기 전에 git status 부터 확인합니다.
- 현재 폴더가 프로젝트 폴더인지 먼저 확인합니다.
- 명령어 결과에 ERROR, CONFLICT, rejected 가 보이면 다음 단계로 넘어가지 않습니다.

REMOTE SETUP

4. GitHub 저장소 만들기와 Collaborator 초대

4-1. GitHub 에서 Repository 만들기

1. GitHub 에서 New repository 를 선택합니다.
2. Repository name 을 입력합니다. 예: team-project
3. Create repository 를 눌러 저장소를 생성합니다.

생성 후 Repository 주소를 복사합니다. 예:

```
https://github.com/USER/team-project.git
```

4-2. 팀원을 Collaborator 로 초대

4. 저장소 Settings 로 이동합니다.
5. Collaborators 메뉴에서 Add people 을 선택합니다.
6. 팀원의 GitHub 계정을 추가합니다.
7. 초대받은 팀원은 invitation 을 승인합니다.

왜 필요한가요?

Collaborator 로 승인되어야 팀원이 공유 Remote Repository 에 branch 를 만들고 push 할 수 있습니다.

팀 규칙 권장

main 에 직접 push 하지 않고, 개인 branch -> Pull Request -> Review -> Merge 순서로 진행하면 실수를 줄일 수 있습니다.

CLONE

5. GitHub 프로젝트를 내 컴퓨터로 가져오기

5-1. git clone

```
git clone https://github.com/USER/team-project.git
```

왜 사용하나요? GitHub의 프로젝트 파일뿐 아니라 Git 이력과 원격 저장소 연결 정보까지 내 컴퓨터에 복제하기 위해 사용합니다.

clone은 최초 1회

처음 프로젝트를 받을 때 clone 합니다. 이미 clone 한 프로젝트를 최신화할 때는 clone 을 다시 하지 않고 pull 을 사용합니다.

5-2. 프로젝트 폴더로 이동

```
cd team-project
```

cd = change directory. 터미널의 현재 위치를 프로젝트 폴더로 이동합니다.

5-3. VS Code 열기

```
code .
```

code = VS Code 실행, . = 현재 폴더. 즉 현재 프로젝트 폴더를 VS Code 로 엽니다.

5-4. Remote 연결 확인

```
git remote -v
```

clone 을 하면 보통 GitHub 저장소가 origin 이라는 이름으로 자동 등록됩니다.

```
origin https://github.com/USER/team-project.git (fetch)
origin https://github.com/USER/team-project.git (push)
```

START WORK

6. 작업 시작: main 최신화 후 내 branch 만들기

6-1. 현재 상태 확인

```
git status
```

git status 는 Git 에서 가장 자주 쓰는 확인 명령입니다. 현재 branch, 수정 파일, 충돌 여부 등을 보여줍니다.

6-2. main 으로 이동

```
git switch main
```

6-3. GitHub 의 최신 main 가져오기

```
git pull origin main
```

왜 하나요? 다른 팀원이 main 에 병합한 최신 내용을 기준으로 내 작업을 시작하기 위해서입니다.

6-4. 내 작업 branch 만들기

```
git switch -c sujin
```

switch = branch 이동, -c = 새 branch 생성. 즉 sujin branch 를 만들고 바로 이동합니다.

```
main (최신 상태)
|
+-- sujin  <- 현재 여기에서 작업
```

6-5. branch 확인

```
git branch
```

* 표시가 현재 위치한 branch 입니다.

중요 습관

새 작업을 시작할 때는 main -> pull -> 새 branch 순서로 시작하면 오래된 코드에서 작업하는 실수를 줄일 수 있습니다.

ADD -> COMMIT -> PUSH

7. 작업한 코드를 기록하고 GitHub 에 올리기

7-1. VS Code 에서 파일 수정

예: hi.txt, main.py, index.html 등을 만들거나 수정합니다.

7-2. 변경사항 확인

```
git status
```

7-3. commit 할 변경사항 준비

```
git add .
```

점(.)은 현재 폴더 아래의 변경사항 전체를 의미합니다. 특정 파일만 추가하려면 git add 파일명 을 사용합니다.

7-4. Local 에 기록

```
git commit -m "로그인 화면 추가"
```

commit 은 Git 이력에 하나의 저장 기록을 만듭니다. 아직 GitHub 로 올라간 것은 아닙니다.

7-5. GitHub 에 첫 push

```
git push -u origin sujin
```

origin = GitHub 저장소, sujin = 올릴 branch. -u 는 Local sujin 과 Remote origin/sujin 을 연결합니다.

7-6. 두 번째부터는

```
git add .
git commit -m "작업 내용"
git push
```

한 세트로 기억

add = 기록할 변경사항 선택 -> commit = Local 에 기록 -> push = GitHub 로 전송

PULL REQUEST

8. Pull Request 와 Merge

push 를 했다고 main 에 자동으로 합쳐지는 것은 아닙니다. GitHub 의 내 branch 에 올라간 상태입니다.

GitHub	
main	기존 코드
sujin	내가 push 한 코드

8-1. Pull Request 만들기

8. GitHub 저장소에서 Compare & pull request 또는 Pull requests -> New pull request 를 선택합니다.
9. base 가 main, compare 가 내 branch 인지 확인합니다.
10. 제목과 작업 내용을 작성하고 Pull Request 를 생성합니다.

```
base: main
compare: sujin
```

뜻: sujin의 변경사항을 main에 합쳐주세요.

8-2. Review 와 Merge

팀원이 변경사항을 확인한 뒤 Approve 하거나 수정 요청을 할 수 있습니다. 승인 후 Merge pull request 를 수행하면 main 에 변경사항이 합쳐집니다.

PR 의 목적

Merge 전에 변경사항을 확인하고, 팀원에게 검토와 허락을 받는 과정입니다.

AFTER MERGE

9. Merge 후 Local main 최신화

GitHub 에서 Merge 가 끝나도 내 컴퓨터의 main 이 자동으로 바뀌는 것은 아닙니다.

```

GitHub main:  A--B--C
               |
               pull
               v
Local main:   A--B--C
  
```

9-1. main 으로 이동

```
git switch main
```

9-2. 최신 main 받기

```
git pull origin main
```

9-3. 다음 작업 시작

```

git switch -c feature-next
# VS Code 에서 작업
git add .
git commit -m "다음 기능 추가"
git push -u origin feature-next
  
```

팀 전체가 해야 하는 일

PR 과 Merge 가 끝난 뒤 다음 작업을 시작하기 전에 모든 팀원이 최신 main 을 pull 하는 습관이 중요합니다.

선택: 사용이 끝난 Local branch 삭제

```
git branch -d sujin
```

삭제 전 해당 작업이 main 에 정상적으로 Merge 됐는지 확인하세요.

LOCAL -> REMOTE

10. 내 컴퓨터에 프로젝트가 먼저 있는 경우

이번에는 반대 상황입니다. 이미 내 컴퓨터에 프로젝트가 있고, 새 GitHub Repository 에 올리고 싶을 때 사용합니다.

```
cd my-project

git init
git add .
git commit -m "Initial commit"

git remote add origin https://github.com/USER/my-project.git
git branch -M main
git push -u origin main
```

각 명령이 하는 일

명령어	왜 사용하는가
git init	현재 폴더를 Git 저장소로 시작
git add .	현재 변경사항을 commit 대상으로 추가
git commit -m "..."	첫 Local commit 생성
git remote add origin URL	GitHub 저장소 주소를 origin 으로 등록
git branch -M main	현재 branch 이름을 main 으로 맞춤
git push -u origin main	Local main 을 GitHub main 으로 처음 전송

clone 과 init 의 차이

GitHub 에 프로젝트가 먼저 있으면 clone. 내 컴퓨터 프로젝트가 먼저 있고 GitHub 에 새로 연결하려면 init + remote add 를 사용합니다.

11. 자주 만나는 오류 1: non-fast-forward

```
! [rejected] sujin -> sujin (non-fast-forward)
Updates were rejected because the tip of your current branch is behind its remote counterpart.
```

의미: GitHub 의 같은 branch 에 내 Local 에는 없는 commit 이 이미 있어서, Git 이 원격 내용을 덮어쓰지 못하도록 push 를 막은 상태입니다.

```
Remote sujin: A--B--C
Local sujin: A--B--D
```

안전하게 확인하는 순서

```
git status
git fetch origin
git pull --rebase origin sujin
```

rebase 가 정상 완료되면:

```
git push -u origin sujin
```

절대 먼저 force push 하지 않기

협업 저장소에서 git push --force 를 무심코 사용하면 다른 팀원의 원격 commit 을 덮어쓸 수 있습니다.

왜 계속 push 해도 해결되지 않나요?

원격 branch 가 Local 보다 앞선 상태는 push 를 반복해도 바뀌지 않습니다. 먼저 원격 변경사항을 가져와 통합해야 합니다.

12. 자주 만나는 오류 2: Conflict / Rebase 진행 중

12-1. Conflict 란?

두 사람이 같은 파일의 같은 부분을 서로 다르게 수정하면 Git 이 자동으로 무엇을 남길지 결정할 수 없습니다.

```
<<<<<<< HEAD
GitHub 쪽 내용
=====
내가 수정한 내용
>>>>>>> my-commit
```

VS Code 에서 최종적으로 남길 내용만 정리하고 <<<<<<<, =====, >>>>>>> 표시를 제거한 뒤 저장합니다.

```
git add 충돌파일명
git rebase --continue
```

12-2. "rebase-merge directory" 오류

이미 rebase 가 진행 중이라는 뜻입니다. 새 rebase 를 시작하지 말고 현재 작업 상태부터 확인합니다.

```
git status
```

충돌을 모두 해결했다면:

```
git add .
git rebase --continue
```

현재 rebase 를 취소하고 원래 상태로 돌아가려면:

```
git rebase --abort
```

12-3. Vim 화면이 갑자기 열렸을 때

커밋 메시지를 확인하라는 편집기 화면일 수 있습니다. 내용 그대로 저장하고 종료하려면:

```
Esc
:wq
Enter
```

문제 해결 기본 원칙

1) git status 확인 2) 화면의 안내 문구 읽기 3) 충돌 해결 4) continue/commit/push 순서로 진행

13. 실무/실습에서 반복하는 하루 작업 사이클

```
# 1) 기준 branch 최신화
git switch main
git pull origin main

# 2) 새 작업 branch 생성
git switch -c feature-login

# 3) VS Code 에서 작업 후 상태 확인
git status

# 4) 기록
git add .
git commit -m "로그인 기능 추가"

# 5) GitHub 에 올리기
git push -u origin feature-login

# 6) GitHub 에서 Pull Request -> Review -> Merge

# 7) Merge 후 다시 최신화
git switch main
git pull origin main
```

한 문장으로 외우기

최신 main 을 pull -> 내 branch 에서 작업 -> add -> commit -> push -> PR -> Merge -> 다시 main pull

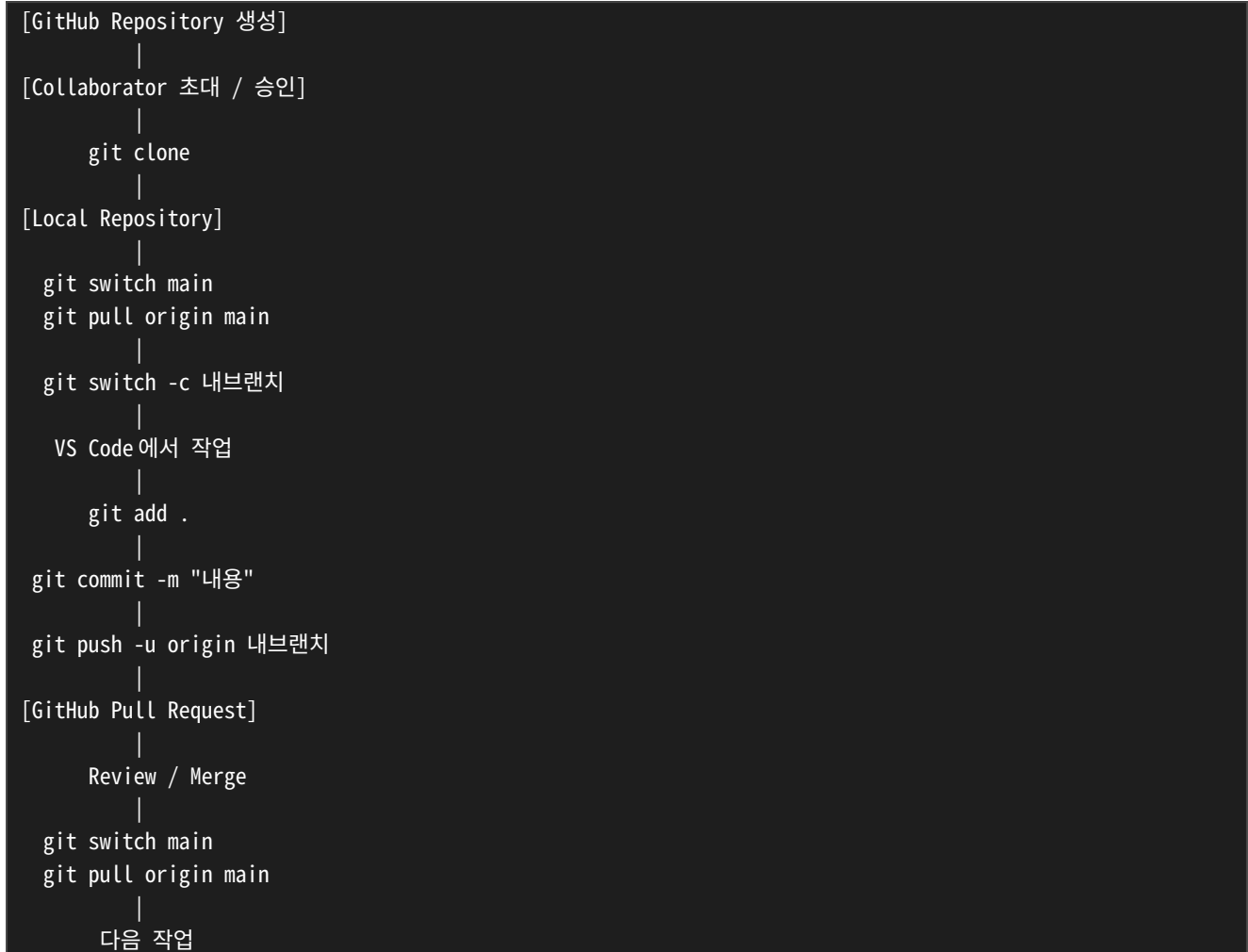
팀 협업 규칙 추천

- main 에서 직접 개발하지 않기
- 작업 시작 전 main pull 하기
- 한 branch 에는 한 가지 작업 목적을 두기
- commit 메시지는 작업 내용을 알아볼 수 있게 쓰기
- push 전에 git status 확인하기
- 같은 파일의 같은 부분을 동시에 수정하지 않도록 역할 나누기

14. 터미널 명령어 치트시트

명령어	의미 / 사용하는 이유
git --version	Git 설치 확인
git status	현재 branch/변경/충돌 상태 확인
git clone URL	GitHub 저장소를 Local 로 복제
cd 폴더명	해당 폴더로 이동
code .	현재 폴더를 VS Code 로 열기
git remote -v	연결된 원격 저장소 확인
git branch	Local branch 목록 확인
git switch main	main 으로 이동
git switch -c 이름	새 branch 생성 + 이동
git pull origin main	GitHub main 최신화
git add .	변경사항 전체를 commit 준비
git commit -m "내용"	Local Git 이력에 기록
git push -u origin 이름	해당 branch 를 처음 Remote 에 연결/전송
git push	연결된 Remote branch 로 전송
git fetch origin	원격 변경 이력을 가져오되 작업 파일에는 바로 병합하지 않음
git rebase --continue	충돌 해결 후 진행 중인 rebase 계속
git rebase --abort	진행 중인 rebase 취소
git branch -d 이름	Local branch 삭제

15. 마지막으로 전체 그림



초보자에게 가장 중요한 습관

명령어를 외우는 것보다 "지금 나는 Local 에 있는가, Remote 에 있는가, 어느 branch 에 있는가"를 먼저 확인하세요. 막히면 git status 부터 확인하면 됩니다.

참고

이 가이드는 제공된 "Git 과 협업" 실습 자료의 Collaborator, Pull Request, Merge, Conflict 흐름을 중심으로 구성하고, 터미널 명령어 의미와 실제 오류 대처법을 초보자 관점에서 보충했습니다.

끝. 이제 직접 한 번 해보면 흐름이 훨씬 빨리 익숙해집니다.